P2137R0
Goals and priorities for C++

Published Proposal, 2020-03-02

This version:
https://wg21.link/p2137r0

Authors:
Chandler Carruth
Timothy Costa
Hal Finkel
Dmitri Gribenko
D. S. Hollman
Chris Kennelly
Thomas Köppe
Damien Lebrun-Grandie
Bryce Adelstein Lelbach
Josh Levenberg
Nevin Liber
Chris Palmer
Tom Scogland
Richard Smith
David Stone
Christian Trott
Titus Winters

Audience:
EWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

This paper describes the goals and priorities which the authors believe make C++ an especially effective programming language for our use cases. That said, our experience, use cases, and needs are

clearly not those of every user. We aren't pushing to directly build consensus on these points. Rather, this is presented as a vehicle to advertise our needs from C++ as a high-performance systems language.

Table of Contents

Historically, C++ has seemed at times to be aligned with the goals and priorities we will outline here—often enough to lend credence to these priorities, but erratically enough to create confusion and friction within the committee and community. We believe that many divisive issues in the committee come from a disagreement on either stated or assumed priorities. As Chandler and Titus said in their joint CppCon talk this year: a programming language is a tool, and different tools are good for different purposes. We think there is great value in priorities that differentiate C++ from the plethora of programming languages, rather than following the crowd and regressing to the mean. But the most important thing is for C++ to state its priorities clearly and explicitly. The knowledge of what to expect would improve the entire community's ability to evaluate and use the language.

We strongly encourage the direction group and committee leadership to lead ongoing discussions on this topic.

§ 1. Language goals

What are our goals for the C++ language? We believe it must support:

1. Performance-critical software
2. Both software and language evolution
3. Code that is simple and easy to read, understand, and write
4. Practical safety guarantees and testing mechanisms
5. Fast and scalable development
6. Current hardware architectures, OS platforms, and environments as they evolve

These are in rough priority order. While there may be nuances where this order does not apply, we generally want to prioritize in this manner.

Performance as the top priority is the defining aspect of C++ for our users. No other programming language provides the performance-critical facilities of C++. This should be the unique value proposition of the C++ programming language.

Overall, these priorities imply a set of long-term goals and are at least somewhat aspirational. Practical concerns, of course, limit our ability to simply or easily achieve these goals, but we think they still describe the direction in which the overall design of C++ should be moving. We feel the design of each and every feature or facility should be rooted in these priorities. Some features and facilities are also directly motivated by these priorities—without them users would be unable to address these goals. However, there is also a broad range of features and facilities (especially domain-focused libraries) which are equally compatible with these priorities and may require some other ranking function to provide roadmap-level prioritization.

Below, we discuss these priorities in more detail to give a deeper understanding of both the nature and motivation of these goals. We also call out non-goals and provide a more detailed outline of the target audience.

§ 1.1. Performance-critical software

All software consumes resources (time, memory, compute, power, etc.), and in many cases raw resource usage is not the biggest concern. Instead, algorithmic efficiency or business logic dominates these concerns. However, there exists software where the rate of resource consumption—its performance—is critical to its successful operation. Another way to think about when performance is critical: would a performance regression be considered a bug for users? Would it even be noticed?

Our goal is to support software where its performance with respect to some set of resource constraints is critical to its successful operation. This overarching goal can be decomposed into a few specific aspects.

Provide the programmer control over every aspect of performance. When faced with some performance problem, the programmer should always have tools within C++ to address it. This does not mean that the programmer is necessarily concerned with ultimate performance at every moment, but in the most constrained scenarios they must be able to “open up the hood” without switching to another language.

Code should perform predictably. The reader (and writer) of code should be able to easily understand its expected performance, given sufficient background knowledge of the environment in which it will run. This need not be precise, but instead can use heuristics and guidelines to avoid surprise. The key priority is that performance, whether good or bad, is unsurprising to users of the C++ language. Even pleasant surprises, when too frequent, can become a problem due to establishing brittle baseline performance that cannot be reliably sustained.

Leave no room for a lower level language. Whether to gain control over performance problems or to gain access to hardware facilities, programmers should not need to leave the rules and structure of the language.

§ 1.2. Both software and language evolution

Titus Winters writes in "Non-Atomic Refactoring and Software Sustainability":

> What is the difference between programming and software engineering? These are nebulous concepts and thus there are many possible answers, but my favorite definition is this: Software engineering is programming integrated over time. All of the hard parts of engineering come from dealing with time: compatibility over time, dealing with changes to underlying infrastructure and dependencies, and working with legacy code or data. Fundamentally, it is a different task to produce a programming solution to a problem (that solves the current [instance] of the problem) vs. an engineering solution (that solves current instances, future instances that we can predict, and - through flexibility - allows updates to solve future instances we may not be able to predict).

From this definition of "software engineering" vs. "programming" we suggest that C++ should prioritize being more of a "software engineering" language, and less of a "programming" language. We specifically are interested in dealing with the time-oriented aspects of software built in this language.

Support maintaining and evolving software written in C++ for decades. The life expectancy of some software will be long and the software will not be static or unchanging in that time. Mistakes will be made and need to be corrected. New functionality will be introduced and old functionality retired and removed. The design of C++ must support and ease every step of this process. This ranges from emphasizing testing and continuous integration to tooling and the ability to make multi-step changes. It also includes constraints on the design of the language itself: we should avoid, or at least minimize, language features that encourage unchangeable constructs. For example, any feature with a contract that cannot be strengthened or weakened without breaking the expected usage patterns is inherently hostile to refactoring. Similarly, features or conventions that require simultaneously updating all users of an API when extending it are inherently hostile towards long-term maintenance of software.

Support maintaining and evolving the language itself for decades. Historically, we have not gotten the design of most language features correct on our first or second try. And we shouldn't expect that to change going forward either. As a consequence, there must be a built-in plan and ability to move C++ forward at a reasonable pace and with a reasonable cost. Simultaneously, an evolving language must not leave software behind to languish, but bring software forward. This requirement should not imply compatibility, but instead some (likely tool-assisted) migratability.

Be mindful of legacy. We support several 100s of millions of lines of C++ code. Globally, there may be as many as 50 billion lines of C++ code. Any evolution of C++ that fails to account for the human investment (training) and legacy code (representing significant capital) is doomed from the start. Note that our _priority_ is restricted to legacy _source code_. Full support, whether across the full language feature set or with full performance, for legacy code beyond that (such as precompiled binaries we call out below) is not a prioritized goal. While that still leaves many options open (such as dedicated and potentially slower features), it does limit the degree to which legacy use cases beyond source code should shape the language design.

§ 1.3. Code that is simple and easy to read, understand, and write

While this is perhaps the least unique (among programming languages) of the goals we list here, and the one most widely shared and discussed in C++'s evolution today, we feel it is important to state it, explain all of what we mean by this, and fit it into our prioritization scheme.

Software, especially at scale and over time, already imposes a burden on engineers due to its complexity. The C++ language should strive for simplicity to reduce the complexity burden on reading, understanding, and writing code. The behavior of code should be easily understood, especially by those unfamiliar with the software system. Consider engineers attempting to diagnose a

serious outage under time pressure—every second spent trying to understand the _language_ is one not spent understanding the _problem_.

While the source code of our software may be read far more often by machines, humans are the most expensive readers (and writers) of software. As a consequence, we need to optimize for human reading, understanding, maintaining, and writing of software, in that order.

Excellent ergonomics. Human capabilities and limitations in the domains of perception, memory, reasoning, and decision-making affect interactions between humans and systems. Ergonomic language design takes human factors into account to increase productivity and comfort, reduces errors and fatigue, making C++ more suitable for humans to use. We can also say that ergonomic designs are accessible to humans. “Readability” is a related, but a more focused concept, connected to only the process of reading code. “Ergonomics” covers all activities where humans interact with C++: reading, writing, designing, discussing, reviewing, and refactoring code, as well as learning and teaching C++. A few examples:

- C++ should not use symbols that are difficult to type, see, or differentiate from similar symbols in commonly used contexts.
- Syntax should be easily parsed and scanned by any human in any development environment, not just a machine or a human aided by semantic hints from an IDE.
- Code with similar behavior should use similar syntax, and code with different behavior different syntax. Behavior in this context should include both the functionality and performance of the code. This is part of conceptual integrity.
- Explicitness must be balanced against conciseness as verbosity and ceremony add cognitive overhead for the reader, while explicitness reduces the amount of outside context the reader must have or assume.
- Ordinary tasks should not require extraordinary care, because humans cannot consistently avoid making mistakes for an extended amount of time.

Support tooling at every layer of the developer experience, including IDEs. The design and implementation of C++ should facilitate both the ease of producing such tools and their effectiveness. Syntax and textual structures that are difficult to recognize and mechanically change without losing meaning should be avoided.

Support software outside of the primary use cases well. There are surprisingly high costs for engineers to switch languages. Even when the primary goal is to support performance-critical software, other kinds of software should not be penalized unnecessarily.

> "The right tool for the job is often the tool you are already using —adding new tools has a higher cost than many people appreciate." —John Carmack

The result of this principle is that there is and will always be a large amount of software written in C++ despite being outside the primary use case. This may be due to familiarity of the engineering team, due to existing libraries written in C++, or due to other ecosystem effects.

Focus on enabling better code patterns rather than restricting bad ones. Adding restrictions to otherwise general facilities can have a disproportionately negative impact in the (possibly rare) cases when they get in the way. Instead, C++ should focus on enabling better patterns, encouraging their use, and creating incentives to ensure people prefer them. The "bad" pattern may be critical for some rare user or some future use case. Put differently, we will not always be able to prevent engineers from writing bad or unnecessarily complex code, and that is okay. We should instead focus on helping reduce the rate that this occurs accidentally, and enabling interface designs, naming patterns, tooling, and diagnostics that warn about dangerous or surprising patterns. This takes the language out of the business of legislating these types of issues.

A concrete example is that we should continue to allow dropping type-system enforced ownership (`std::unique_ptr::release`) for the edge cases where this is important. But we can and should ensure that using type-system enforced ownership is the easiest (and ideally default) approach when allocating memory.

The behavior and semantics of code should be clearly and simply specified whenever possible. Leaving behavior undefined in some cases for invalid, buggy, or non-portable code may be necessary but comes at a high cost and should be avoided whenever other priorities (such as performance) allow. Every case where behavior is left undefined should be clearly spelled out with a strong rationale for this tradeoff. The code patterns without defined behavior should be teachable and understandable by engineers. And finally, there must be mechanisms available to detect undefined behavior, at best statically, and at worst dynamically with high probability and at minimal cost.

Adhere to the principle of least surprise. Defaults should match typical usage patterns. Implicit features should be unsurprising and expected, while explicit syntax should inform the reader about any behavior which might otherwise be surprising. The core concepts of implicit vs. explicit syntax are well articulated in the Rust community, despite some specific examples and conclusions not necessarily adhering to this principle.

Design features to be simple to implement. Syntax, structure, language, and library features should be chosen while keeping the complexity of the implementation manageable. This reduces bugs, and will in most cases make the features easier to understand.

§ 1.4. Practical safety guarantees and testing mechanisms

Our goal is to add as much language-level safety and security to C++ as possible when balanced against the pragmatic need for software performance, programmer ergonomics, and continued support of existing/legacy C and C++ code. This results in a hybrid strategy where we prove as much safety as we can (within these constraints) at compile time, and combine this with dynamic runtime checking and a strong testing methodology ranging from unit tests through integration and system tests all the way to coverage-directed fuzz testing. We have specific criteria that are important for this strategy to be successful:

Make unsafe or risky aspects of an operation, interface, or type explicit and syntactically visible. This will allow the software to use the precise flexibility needed and to minimize its exposure, while still aiding the reader. It can also help the reader more by indicating the specific nature of risk faced by a given construct. More simply, safe things shouldn't look like unsafe things and unsafe things should be easily recognized when reading code.

Common patterns of unsafe or risky code must support static checking. Waiting until a dynamic check is too late for the 80% case. A canonical example here are thread-safety annotations for basic mutex lock management to allow static checking. This handles the common patterns, and we use dynamic checks (TSan, deadlock detection) to handle edge cases.

All unsafe or risky operations and interfaces must support some dynamic checking. Users need _some_ way to test and verify that their code using any such interface is in fact correct. Uncheckable unsafety removes any ability for the user to gain confidence. This means we need to design features with unsafe or risky aspects with dynamic checking in mind. A concrete example of this can be seen in facilities that allow indexing into an array: such facilities should be designed to have the bounds of the array available to implement bounds checking when desirable. These dynamic checks may even need to be designed using high-cost techniques that only apply to continuous integration and testing (such as Sanitizers and shadow memory), but the key thing is that they are available in some form to verify whether the underlying code is correct.

§ 1.5. Fast and scalable development

Engineers interact with many language tools that need different levels of processing. IDEs and editor tools often use minimal parsing to give rapid feedback. Engineers will also iterate repeatedly on any compile error. Building, testing, and debugging complete the "edit, test, debug" cycle that is the critical path of software development iteration. Each step needs to be fast and scalable. Raw speed is essential for small projects and local development. Scalability is necessary to address the large software systems we currently use.

Syntax should parse with bounded, small look-ahead. Using syntax that requires unbounded look-ahead or fully general backtracking adds significant complexity to parsing and makes it harder to provide high quality error messages. The result is both slower iteration and more iterations, a multiplicative negative impact on productivity. Humans aren't immune either and can be confused by constructs that appear to mean one thing but actually mean another. Instead, we should design for syntax that is fast to parse, with easy and reliable error messages.

No semantic or contextual information used when parsing. The more context, and especially the more _semantic_ context, required for merely parsing code, the fewer options available to improve the performance of tools and compilation. Cross-file context has an especially damaging effect on the potential distributed build graph options. Without these options, we will again be unable to provide fast programmer iteration as the codebase scales up.

Support separate compilation, including parallel and distributed strategies. We cannot assume coarse-grained compilation without blocking fundamental scalability options for build systems of large software.

§ 1.6. Modern hardware architectures, OS platforms, and environments

We of course continue to care that C++ supports all of the major, modern platforms, the hardware architectures they run on, and the environments in which their software runs. However, this only forms a fairly small and focused subset of the platforms that have historically been supported and motivated specific aspects of the C++ design. We would suggest this be narrowed as much as possible. An initial non-exhaustive list of platforms we believe should continue to be prioritized:

- Linux
- Android
- Windows
- macOS, iOS, watchOS, tvOS
- Fuchsia
- WebAssembly
- OS/kernel
- QNX
- Bare-metal

Similarly, we should prioritize support for 64-bit little-endian hardware, including:

- x86-64
- AArch64 (ARM 64-bit)
- PPC64LE (Power ISA, 64-bit, Little Endian)
- RV64I (RISC-V 64-bit)

We believe C++ should continue to strive to support some GPUs, other restricted computational hardware and environments, and embedded environments, although likely not all historical platforms of this form. While this should absolutely include future and emerging hardware and platforms, those shouldn't _disproportionately_ shape the fundamental library and language design -- they remain relatively new and narrow in user base at least initially.

We do not need to prioritize support for historical platforms. To use a hockey metaphor, C++ should not skate to where the puck is, much less where the puck was twenty years ago. We have existing systems to support those platforms where necessary. Instead, C++ should be forward-leaning in its platform support. To give a non-exhaustive list of example, we should not prioritize support for:

- Byte sizes other than 8-bits, or non-power-of-two word sizes
- Source code encodings other than UTF-8
- Big- or mixed-endian (at least for computation; accessing encoded data remains useful)
- Non-2's-complement integer formats (we supported C++20 dropping support for this)
- Non-IEEE 754 floating point format as default floating point types
- Source code in file systems that don't support file extensions or nested directories

A specific open question is whether supporting 32-bit hardware and environments (including for example x32) is an important goal. While some communities, especially in the embedded space, currently rely on this, it comes at a significant cost and without significant advantage to other communities. We would like to attempt to address the needs of these communities within a 64-bit model to simplify things long-term, but this may prove impossible.

§ 2. Non-goals

There are common or expected goals of many programming languages that we explicitly call out as non-goals for the C++ language from our perspective. That doesn't make these things bad in any way, but reflects the fact that they do not provide meaningful value to us and come with serious costs and/or risks.

§ 2.1. Stable language and library ABI

We would prefer to provide better, dedicated mechanisms to decompose software subsystems in ways that scale over time rather than providing a stable ABI across the language and libraries. Our experience is that providing broad ABI-level stability for high-level constructs is a significant and permanent burden on their design. It becomes an impediment to evolution, which is one of our stated goals.

This doesn't preclude having low-level language features or tools to create specific and curated stable ABIs (or even serializable protocols). Using any such facilities will also cause developers to explicitly state where they are relying on ABI and isolating it (in source) from code which does not need that stability. However, these facilities would only expose a restricted set of language features to avoid coupling the high-level language to particular stabilized interfaces. There is a wide range of such facilities that should be explored, from serialization-based systems like protobufs or pickling in Python, to COM or Swift's "resilience" model. The specific approach should be designed specifically around the goals outlined above in order to fit the C++ language.

§ 2.2. Backwards or forwards compatibility

Our goals are focused on _migration_ from one version of C++ to the next rather than _compatibility_ between them. This is rooted in our experience with evolving software over time more generally and a live-at-head model. Any transition, whether based on backward compatibility or a migration plan, will require some manual intervention despite our best efforts, due to Hyrum's Law, and so we should acknowledge that upgrades require active migrations.

§ 2.3. Legacy compiled libraries without source code or ability to rebuild

We consider it a non-goal to support legacy code for which the source code is no longer available, though we do sympathize with such use cases and would like to see tooling mentioned above allow easier bridging between ABIs in these cases. Similarly, plugin ABIs aren't our particular concern, yet we're interested in seeing tooling which can help bridge between programs and plugins which use different ABIs.

§ 2.4. Support for existing compilation and linking models

We are willing to change the compilation and linking model as necessary to achieve these goals. Compilation models and linking models should be designed to suit the needs of C++ and its use cases,

tools, and environments, not what happens to have been implemented thus far in compilers and linkers.

A concrete example of this non-goal: it means platforms that cannot update their compiler and linker when updating the C++ language are not supported.

§ 3. Prioritizing across domain-motivated features

Any language or library facility should be prioritized first and foremost according to the top-level language goals above, and how well the facility meets those goals. However, many facilities or features will meet these criteria equally well. These features often serve different domains or users of the language, and we still need an effective way to prioritize them. This prioritization is significantly different from everything before as it doesn't impact the specific design of the feature itself, and is only intended to guide how equally well-designed facilities are prioritized for incremental enhancement of the language.

At this stage, the primary prioritization should be based on the relative cost-benefit ratio of the feature. The cost is a function of the effort required to specify and implement the feature. The benefit is the number of impacted users and the magnitude of that impact. We don't expect to have concrete numbers for these, but we expect prioritization decisions between features to be expressed using this framework.

Secondarily, priority should be given based on effort: both effort already invested and effort ready to commit to the feature. This should not overwhelm the primary metric, but given two equally impactful features we should focus on the one that is moving fastest.

§ 4. Acknowledgements

Many people contributed suggestions, ideas, and helped create this document but may not completely agree with the exact set of goals and their priorities. We'd like to credit their help writing it without them feeling pressured to sign onto such a specific position, and so some are listed here:

- Jeff Snyder
- Lee Howes
- Louis Brandy
- David Goldblatt
- Michael Spencer

That said, this is an incomplete list and we thank all the others who did significant work to help arrive at this document.